Developer Workflow Cookbook: Conda + uv (Corrected)

Updated January 10, 2026

This cookbook describes a repeatable workflow for using Conda for the base environment and system/tooling packages (Python, OpenSSL, SQLite, CA certificates, JupyterLab, etc.) and uv for Python dependency management (pyproject.toml + uv.lock) while installing domain packages into the active Conda environment.

Important correction: Do **not** use **uv pip sync** against a shared Conda environment that contains packages you want to keep (e.g., JupyterLab). **uv pip sync** removes packages not listed in the requirements file. Use **uv pip install** to apply locked requirements additively without uninstalling your Conda tooling.

Key files

environment.yaml - creates the Conda environment (runtime + system/tooling deps)

pyproject.toml - declares Python dependencies (what you want)

uv.lock - locks resolved Python dependencies (what you actually get)

requirements.locked.txt - exported pinned requirements used to install into Conda (additively)

1. Template: environment.yaml

Prefer conda-forge for consistent builds. Keep Conda dependencies focused on Python itself, system libraries, and developer tooling (including Jupyter). Let uv manage most domain-specific Python packages.

```
name: YOUR_ENV_NAME
channels:
  - conda-forge
  - defaults
dependencies:
  # Core runtime
  - python>=3.13
  - pip
  - setuptools
  - wheel

  # System / shared libraries
  - openssl
  - ca-certificates
  - sqlite

  # Developer tooling
  - uv
  - git
  - curl

  # Jupyter + kernel support (owned by Conda)
  - jupyterlab
  - ipykernel
  - ipython
```

Notes: If you need platform-specific system packages, keep those in Conda. Avoid a large pip section in environment.yaml if uv is your dependency manager.

2. Initial setup from scratch (Conda + uv)

Step 0 - Prerequisites

Install Conda (or Miniforge/Mambaforge) and ensure conda (or mamba) is available in your shell.

Step 1 - Create the Conda environment

```
# from the repo root
conda env create -f environment.yaml
# or name override:
conda env create -f environment.yaml -n YOUR_ENV_NAME
# mamba equivalent:
mamba env create -f environment.yaml
```

Step 2 - Activate and verify

```
conda activate YOUR_ENV_NAME
python -c "import sys; print(sys.executable)"
# should point inside your CONDA_PREFIX
```

Step 3 - Initialize uv in the repo (only once per repo)

If the repo already has pyproject.toml, skip initialization.

```
# create a minimal pyproject.toml (if not present)
uv init --bare

# ensure pyproject.toml includes:
# [project]
# requires-python = ">=3.13"
```

Step 4 - Add Python dependencies with uv

Add runtime and dev dependencies using uv. This updates pyproject.toml and uv.lock.

```
# examples
uv add requests
uv add numpy pandas
uv add --dev ruff pytest
```

Step 5 - Lock dependencies

```
uv lock
```

Step 6 - Export pinned requirements for Conda

Export a fully pinned, reproducible set of requirements that pip-style tools can install. This file is an installation artifact; it does not modify the environment by itself.

```
uv pip compile pyproject.toml -o requirements.locked.txt
```

Step 7 - Install locked requirements into the active Conda environment (safe)

Apply the pinned set into the currently active Conda environment **without uninstalling** packages that are not listed (e.g., JupyterLab). This is the safe approach for a shared Conda environment.

```
# optional: preview what would change (recommended in shared envs)
uv pip install -r requirements.locked.txt --dry-run

# install/upgrade/downgrade only what's needed; does NOT remove unrelated packages
uv pip install -r requirements.locked.txt
```

Warning: If a package in requirements.locked.txt overlaps with something Conda installed (e.g., numpy/protobuf), uv may replace that package version in site-packages to satisfy the lock. It will not remove unrelated tooling packages unless you use uv pip sync.

Step 8 - Register the kernel for JupyterLab (optional)

```
python -m ipykernel install --user --name YOUR_ENV_NAME --display-name "Python (YOUR_ENV_NAME)"
```

What to commit to git

Commit: environment.yaml, pyproject.toml, uv.lock, and (if used for CI/deploy) requirements.locked.txt. Do not commit: your local Conda prefix, caches, or platform-specific build artifacts.

3. When you add or change a Python dependency later

Use this flow anytime after initial setup when you need to add, remove, or upgrade a Python dependency in a repo.

Step 1 - Activate the shared Conda environment

```
conda activate YOUR_ENV_NAME
```

Step 2 - Add/remove/update dependencies using uv (in that repo)

```
# add a new runtime dependency
uv add rich

# add a dev dependency
uv add --dev mypy

# remove a dependency
uv remove rich
```

Step 3 - Re-lock

```
uv lock
```

Step 4 - Re-export the pinned requirements file

```
uv pip compile pyproject.toml -o requirements.locked.txt
```

Step 5 - Install locked requirements into the shared Conda environment (safe)

```
uv pip install -r requirements.locked.txt --dry-run
uv pip install -r requirements.locked.txt
```

Step 6 - Sanity check

```
python -c "import sys; print(sys.version)"
python -c "import pkgutil; print('ok')"
# or
pytest -q
```

4. Guidance for one shared Conda environment across multiple repos

If you share a single Conda environment across multiple repos (e.g., langchain, langgraph, langflow), treat each repo's lockfile as **repo-specific**. Installing Repo A's locked requirements may change versions that Repo B relies on, and vice versa. Because of that, avoid any command that tries to make the environment match one repo exactly.

Do:

• Use **uv pip install -r requirements.locked.txt** (additive; no uninstalls).

• Keep Jupyter/JupyterLab/ipykernel in Conda.

• Consider aligning versions for high-conflict libraries (pydantic, fastapi, numpy, protobuf, httpx).

Don't:

• Do not run **uv pip sync** in the shared Conda environment.

• Do not assume one repo's lockfile can represent the entire shared environment.

Optional improvement: shared constraints

If conflicts become frequent, maintain a shared constraints file (e.g., requirements.shared.locked.txt) that pins common foundation libraries. Then install each repo with -c to keep them aligned:

```
uv pip install -r requirements.locked.txt -c requirements.shared.locked.txt
```

5. Troubleshooting quick hits

uv installs into the wrong Python:

```
python -c "import sys; print(sys.executable)"
# ensure it points inside your CONDA_PREFIX after 'conda activate'
```

Binary wheels fail to build:

Prefer Conda packages for compiled libraries or add missing system deps to environment.yaml.

Need a clean slate:

Remove and recreate the Conda env from environment.yaml, then rerun: uv lock -> uv pip compile -> uv pip install.